

OPTIMIZACIÓN DE TAREAS

automatización en sistemas Linux



CRON

UBUNTU SERVER

INSTALACIÓN Y CONFIGURACIÓN DE CRON

Instalación de CRON

Actualizamos nuestro Linux

```
victor@victor:~$ sudo apt update
[sudo] password for victor:
Obj:1 http://security.ubuntu.com/ubuntu plucky-security InRelease
Obj:2 http://es.archive.ubuntu.com/ubuntu plucky InRelease
Obj:3 http://es.archive.ubuntu.com/ubuntu plucky-updates InRelease
Obj:4 http://es.archive.ubuntu.com/ubuntu plucky-backports InRelease
Des:5 http://es.archive.ubuntu.com/ubuntu plucky/main Translation-es [325 kB]
]
Des:6 http://es.archive.ubuntu.com/ubuntu plucky/restricted Translation-es [816 B]
Des:7 http://es.archive.ubuntu.com/ubuntu plucky/universe Translation-es [1.366 kB]
Des:8 http://es.archive.ubuntu.com/ubuntu plucky/multiverse Translation-es [61,9 kB]
Descargados 1.754 kB en 1s (2.040 kB/s)
Se pueden actualizar 62 paquetes. Ejecute «apt list --upgradable» para verlos.
victor@victor:~$ sudo apt upgrade
Los paquetes indicados a continuación se instalaron de forma automática y ya no son necesarios.
  libutempter0                                linux-modules-extra-6.14.0-15-generic
  linux-headers-6.14.0-15                     linux-tools-6.14.0-15
  linux-headers-6.14.0-15-generic             linux-tools-6.14.0-15-generic
  linux-modules-6.14.0-15-generic
Utilice «sudo apt autoremove» para eliminarlos.

Upgrading:
  apparmor                                     mount
```

Instalamos CRON y comprobamos que se ha instalado correctamente

```
victor@victor:~$ sudo apt install cron
cron ya está en su versión más reciente (3.0pl1-194ubuntu1).
fijado cron como instalado manualmente.
Los paquetes indicados a continuación se instalaron de forma automática y ya no son necesarios.
  libutempter0                                linux-modules-extra-6.14.0-15-generic
  linux-headers-6.14.0-15                     linux-tools-6.14.0-15
  linux-headers-6.14.0-15-generic             linux-tools-6.14.0-15-generic
  linux-modules-6.14.0-15-generic
Utilice «sudo apt autoremove» para eliminarlos.

Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 2
victor@victor:~$ |

victor@victor:~$ dpkg -l | grep cron
ii  cron                    3.0pl1-194ubuntu1
    amd64                  process scheduling daemon
ii  cron-daemon-common      3.0pl1-194ubuntu1
    all                    process scheduling daemon's configuration files
victor@victor:~$
```

Habilitamos el servicio de CRON

Con `sudo systemctl enable cron`, y `sudo systemctl start cron`, iniciamos el servicio cron y lo habilitamos para que arranque siempre con la maquina

```
victor@victor:~$ sudo systemctl enable cron
Synchronizing state of cron.service with SysV service script with /usr/lib/s
ysystemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable cron
victor@victor:~$ sudo systemctl start cron
victor@victor:~$ sudo systemctl status cron
● cron.service - Regular background program processing daemon
   Loaded: loaded (/usr/lib/systemd/system/cron.service; enabled; preset:~>
   Active: active (running) since Mon 2025-12-15 07:38:56 UTC; 6min ago
   Invocation: b19fd83954494712a7c44f2e684265b8
      Docs: man:cron(8)
    Main PID: 979 (cron)
       Tasks: 1 (limit: 1873)
      Memory: 476K (peak: 1.9M)
         CPU: 14ms
    CGroup: /system.slice/cron.service
            └─979 /usr/sbin/cron -f -P

dic 15 07:38:56 victor systemd[1]: Started cron.service - Regular backgroun>
dic 15 07:38:56 victor (cron)[979]: cron.service: Referenced but unset envi>
dic 15 07:38:56 victor cron[979]: (CRON) INFO (pidfile fd = 3)
dic 15 07:38:56 victor cron[979]: (CRON) INFO (Running @reboot jobs)
dic 15 07:45:01 victor CRON[11474]: pam_unix(cron:session): session opened >
dic 15 07:45:01 victor CRON[11475]: (root) CMD (command -v debian-sa1 > /de>
dic 15 07:45:01 victor CRON[11474]: pam_unix(cron:session): session closed >
lines 1-19/19 (END)
```

Configurar CRON

Con `crontab -e` accedemos al archivo de configuración de cron, en el que podemos programar tareas

```
victor@victor:~$ crontab -e
no crontab for victor - using an empty one
Select an editor. To change later, run select-editor again.
 1. /bin/nano          <---- easiest
 2. /usr/bin/vim.basic
 3. /usr/bin/vim.tiny
 4. /bin/ed

Choose 1-4 [1]: 1
```

Agregamos tareas en el formato que nos indican los comentarios del archivo. Para comprobar el funcionamiento, programamos la creación de un directorio de prueba

```
GNU nano 8.3 /tmp/crontab.LLRILF/crontab *
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
5 8 * * * /bin/mkdir /home/victor/cron_test
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h dom mon dow  command
```

Como podemos comprobar, el directorio se ha creado a la hora programada de manera satisfactoria

```
victor@victor:~$ ls
victor@victor:~$ date
mié 17 dic 2025 08:04:21 UTC
victor@victor:~$ ls
cron_test
victor@victor:~$ date
mié 17 dic 2025 08:05:10 UTC
victor@victor:~$
```

Programar un respaldo diario

Cron nos permite ejecutar nuestros propios scripts. Vamos a crear un script para hacer un respaldo de un directorio y dejar registro en un log. Creamos el archivo [backup.sh](#) y codificamos el siguiente script

```
GNU nano 8.3          New Buffer *
#!/bin/bash

rsync -av /home/victor/documentos /home/victor/respaldo

echo "Respaldo completado el $(date)" >> /home/victor/respaldo/respaldo.log

victor@victor:~$ chmod +x backup.sh
victor@victor:~$ ls -l backup.sh
-rwxrwxr-x 1 victor victor 147 dic 17 08:08 backup.sh
victor@victor:~$ |
```

Le damos permisos de ejecución para que pueda ejecutarse nuestro script

Ahora programamos la tarea en cron para que todos los días a las 2 AM ejecute nuestro script de respaldo

```
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
0 2 * * * /bin/bash /home/victor/backup.sh
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h dom mon dow  command
```

Creamos el directorio documentos y algunos archivos para comprobar que nuestra tarea programada se ejecuta correctamente

```
victor@victor:~/documentos$ ls
for_delete  important_file  test  trash
victor@victor:~/documentos$
```

Como se puede ver en la siguiente imagen más abajo, con `crontab -l` podemos ver que tareas tenemos programadas. En este caso cambian las horas y minutos respecto a la primera captura con el fin de poder comprobar que la tarea se ejecuta sin tener que esperar a las 2 AM

```

victor@victor:~$ crontab -l
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
13 8 * * * /bin/bash /home/victor/backup.sh
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h  dom mon dow   command
victor@victor:~$

```

Una vez llegada la hora de la tarea, vemos que esta se ejecuta perfectamente.

```

victor@victor:~$ ls -l documentos/
total 0
-rw-rw-r-- 1 victor victor 0 dic 17 08:10 for_delete
-rw-rw-r-- 1 victor victor 0 dic 17 08:11 important_file
-rw-rw-r-- 1 victor victor 0 dic 17 08:10 test
-rw-rw-r-- 1 victor victor 0 dic 17 08:10 trash
victor@victor:~$ ls
backup.sh  cron_test  documentos
victor@victor:~$ ls
backup.sh  cron_test  documentos  respaldo
victor@victor:~$ ls respaldo/
documentos  respaldo.log
victor@victor:~$ ls -l respaldo/
total 8
drwxrwxr-x 2 victor victor 4096 dic 17 08:11 documentos
-rw-rw-r-- 1 victor victor  53 dic 17 08:13 respaldo.log
victor@victor:~$ ls -l respaldo/documentos/
total 0
-rw-rw-r-- 1 victor victor 0 dic 17 08:10 for_delete
-rw-rw-r-- 1 victor victor 0 dic 17 08:11 important_file
-rw-rw-r-- 1 victor victor 0 dic 17 08:10 test
-rw-rw-r-- 1 victor victor 0 dic 17 08:10 trash
victor@victor:~$

```

Si hacemos un cat a nuestro archivo de log podemos ver que el respaldo se registro correctamente

```
victor@victor:~$ cat respaldo/respaldo.log
Respaldo completado el mié 17 dic 2025 08:13:01 UTC
victor@victor:~$
```

Limpiar archivos temporales semanalmente

Creamos y hacemos ejecutable nuestro script de limpieza

```
GNU nano 8.3 limpiar_tmp.sh *
#!/bin/bash

find /tmp -type f -mtime +7 -exec rm {} \;
echo "Limpieza completada el $(date)" >> /home/victor/limpieza.log
```

```
victor@victor:~$ ls
backup.sh  cron_test  documentos  limpiar_tmp.sh  respaldo
victor@victor:~$
```

Programamos la tarea para que limpie de archivos el directorio /tmp todos los domingos a las 3 AM

```
victor@victor:~$ crontab -l
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
22 8 * * 7 /bin/bash /home/victor/limpiar_tmp.sh
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h  dom mon dow   command
victor@victor:~$
```

Nuevamente la hora cambia para poder comprobar su funcionamiento. Aunque el día aparezca como 7, fue cambiado a todos los días con el mismo fin de comprobar el funcionamiento.

Creamos un archivo de texto en /tmp para comprobar el funcionamiento

```
victor@victor:~$ ls -l /tmp
total 0
drwx----- 2 root    root    40 dic 17 07:52 snap-private-tmp
drwx----- 3 root    root    60 dic 17 08:11 systemd-private-a5d9c2460a704c9ba
fe8e2e6cda60596-fwupd.service-IqteDw
drwx----- 3 root    root    60 dic 17 07:52 systemd-private-a5d9c2460a704c9ba
fe8e2e6cda60596-ModemManager.service-f0Ga92
drwx----- 3 root    root    60 dic 17 07:52 systemd-private-a5d9c2460a704c9ba
fe8e2e6cda60596-polkit.service-HM5CoE
drwx----- 3 root    root    60 dic 17 07:52 systemd-private-a5d9c2460a704c9ba
fe8e2e6cda60596-systemd-logind.service-4ilBsJ
drwx----- 3 root    root    60 dic 17 08:11 systemd-private-a5d9c2460a704c9ba
fe8e2e6cda60596-upower.service-2Nl8V0
-rw-rw-r-- 1 victor  victor    0 dic 17 08:20 test_file.txt
victor@victor:~$ date
mié 17 dic 2025 08:24:45 UTC
victor@victor:~$
```

A la hora prevista comprobamos la ejecución de la tarea. La tarea se ejecuto, pero el archivo no se borró. Esto se debe a que se necesitan privilegios de sudo para borrar archivos en /tmp, y en nuestro script no fueron configurados correctamente. Sin embargo como vemos, el log se creo sin ningun tipo de problema, comprobando que la tarea se ejecuta satisfactoriamente

```
victor@victor:~$ date
mié 17 dic 2025 08:27:04 UTC
victor@victor:~$ ls -l /tmp
total 0
drwx----- 2 root    root    40 dic 17 07:52 snap-private-tmp
drwx----- 3 root    root    60 dic 17 08:11 systemd-private-a5d9c2460a704c9ba
fe8e2e6cda60596-fwupd.service-IqteDw
drwx----- 3 root    root    60 dic 17 07:52 systemd-private-a5d9c2460a704c9ba
fe8e2e6cda60596-ModemManager.service-f0Ga92
drwx----- 3 root    root    60 dic 17 07:52 systemd-private-a5d9c2460a704c9ba
fe8e2e6cda60596-polkit.service-HM5CoE
drwx----- 3 root    root    60 dic 17 07:52 systemd-private-a5d9c2460a704c9ba
fe8e2e6cda60596-systemd-logind.service-4ilBsJ
drwx----- 3 root    root    60 dic 17 08:11 systemd-private-a5d9c2460a704c9ba
fe8e2e6cda60596-upower.service-2Nl8V0
-rw-rw-r-- 1 victor  victor    0 dic 17 08:20 test_file.txt
victor@victor:~$ ls
backup.sh  cron_test  documentos  limpiar_tmp.sh  limpieza.log  respaldo
```

Supervisar disco y enviar alertas

Creamos un script que compruebe el espacio usado en disco, y que nos enviara una alerta a nuestro email si este supera el 80%.

Primero instalamos mailutils para poder enviar el email

```
victor@victor:~$ sudo apt install mailutils
Los paquetes indicados a continuación se instalaron de forma automática y ya
no son necesarios.
  libutempter0                                linux-modules-extra-6.14.0-15-generic
  linux-headers-6.14.0-15                     linux-tools-6.14.0-15
  linux-headers-6.14.0-15-generic             linux-tools-6.14.0-15-generic
  linux-modules-6.14.0-15-generic
Utilice «sudo apt autoremove» para eliminarlos.

Installing:
  mailutils

Installing dependencies:
  gsasl-common      libgssglue1      libmysqlclient24  mailutils-common
  guile-3.0-libs    libidn12        libnsl2           mysql-common
  libgc1            libltdl7        libntlm0          postfix
  libgsasl18        libmailutils9t64 libpq5            ssl-cert
```

Ahora creamos nuestro script y lo hacemos ejecutable

```
GNU nano 8.3                                supervisor_disco.sh *
#!/bin/bash

USO=$(df / | grep / | awk '{print $5}' | sed 's/%//')

if [ $USO -gt 80 ]; then
echo "El uso del disco supera el 80%. Uso actual: $USO%" | mail -s "alerta de espacio en disco" vcp0006@alu.medac.es
fi
```

```
victor@victor:~$ chmod +x supervisor_disco.sh
victor@victor:~$ ls -l supervisor_disco.sh
-rwxrwxr-x 1 victor victor 211 dic 17 08:33 supervisor_disco.sh
victor@victor:~$
```

Finalmente programamos la tarea

```
GNU nano 8.3                                /tmp/crontab.l4dN4b/crontab *
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
0 * * * * /bin/bash /home/victor/supervisor_disco.sh
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h dom mon dow  command
```